

Towards Engineering Multi-Agent LLMs: A Protocol-Driven Approach

Zhenyu Mao¹, Jacky Keung¹, Fengji Zhang¹, Shuo Liu^{1,*}, Yifei Wang¹, and Jialong Li²

¹City University of Hong Kong, Hong Kong, China ²Waseda University, Tokyo, Japan

zhenyumao2-c@my.cityu.edu.hk, Jacky.Keung@cityu.edu.hk, fzhong278-c@my.cityu.edu.hk

swang4748-c@my.cityu.edu.hk, lijialong@fuji.waseda.jp

*corresponding author: sliu273-c@my.cityu.edu.hk

Abstract—The increasing demand for software development has driven interest in automating software engineering (SE) tasks using Large Language Models (LLMs). Recent efforts extend LLMs into multi-agent systems (MAS) that emulate collaborative development workflows, but these systems often fail due to three core deficiencies: under-specification, coordination misalignment, and inappropriate verification, arising from the absence of foundational SE structuring principles. This paper introduces Software Engineering Multi-Agent Protocol (SEMAP), a protocol-layer methodology that instantiates three core SE design principles for multi-agent LLMs: (1) explicit behavioral contract modeling, (2) structured messaging, and (3) lifecycle-guided execution with verification, and is implemented atop Google’s Agent-to-Agent (A2A) infrastructure. Empirical evaluation using the Multi-Agent System Failure Taxonomy (MAST) framework demonstrates that SEMAP effectively reduces failures across different SE tasks. In code development, it achieves up to a 69.6% reduction in total failures for function-level development and 56.7% for deployment-level development. For vulnerability detection, SEMAP reduces failure counts by up to 47.4% on Python tasks and 28.2% on C/C++ tasks.

Index Terms—Large Language Models, Multi-Agent Systems, AI Agent Protocols, Software Engineering, AI for SE

I. INTRODUCTION

As software has become an essential backbone supporting almost every aspect of modern society, nowadays there has been a rapid increase in demand not only for more software but also for more advanced software development to sustain and accelerate technological progress. However, although the number of software developers is reaching 47 million in 2025 [1], this remains insufficient to meet the rising demand, prompting increasing reliance on AI-powered tools.

Large Language Models (LLMs) have emerged as a promising solution to address the growing gap between software development demand and available developer resources. With LLMs’ strong ability to understand and interpret both natural and machine languages, LLMs have been successfully applied across a wide range of domains, including evolutionary computation [2], communication analysis [3], self-adaptive systems [4], and, increasingly, software engineering (SE) [5]–[7]. Recent research has explored how teams of LLMs can be orchestrated to simulate collaborative workflows in real-world SE tasks [8] such as requirement engineering [9], [10], code generation [11], [12], quality assurance [13], [14], and maintenance [15], [16]. In addition, several end-to-end multi-

agent LLM frameworks, such as AutoGen, MetaGPT, and ChatDev, have been developed to model distinct development roles and workflows based on existing software process models such as Agile [17] and Waterfall [18].

Despite their conceptual appeal, current multi-agent LLM systems often under-perform in practice, as evidenced by high failure rates on SE tasks. Cemri et al. [19] introduced the Multi-Agent System Failure Taxonomy (MAST) and identified three recurring issue categories: under-specification, inter-agent misalignment, and inappropriate verification.

From a classical SE perspective, these failures can be interpreted as symptoms of three deeper structural deficiencies: (1) inadequate component design, where agent responsibilities and role boundaries are poorly defined; (2) insufficient interface specification, where inter-agent communication lacks semantic structure or typed formats; and (3) inappropriate transition logic, where the system progresses between stages without formal gating or validation. Traditional SE addresses such challenges through well-established principles: components are designed with explicit contracts, interfaces are formalized to ensure consistent integration, and system behavior is governed by state-based coordination models that enforce correctness through verification. However, these SE principles remain largely absent from current multi-agent LLM frameworks, which are often built on loosely coupled prompts and informal, ad hoc role specifications.

To operationalize this vision, this paper introduces the Software Engineering Multi-Agent Protocol (SEMAP), a protocol-layer methodology that instantiates three core SE-inspired design principles: (1) explicit behavioral contract modeling, (2) structured messaging, and (3) lifecycle-guided execution with verification. SEMAP is implemented as a lightweight protocol middleware atop Google’s Agent-to-Agent (A2A) infrastructure and supports both centralized and decentralized workflows. While elements of these principles have appeared independently, SEMAP is the first to unify them in a domain-specialized protocol for the coordination in multi-agent LLMs.

II. BACKGROUND

A. Multi-Agent LLMs for SE

The recent progress in LLMs has led the development of multi-agent LLMs, where multiple LLM instances, each assuming a distinct, role-specific function, collaborate to solve

complex tasks. This approach is inspired by human collaborative work, in which modular responsibilities are distributed among team members and coordinated through structured communication. In the context of SE, multi-agent LLMs seek to automate SE processes by breaking down high-level goals into sub-tasks and delegating them to specialized agents.

Multi-agent LLMs have been utilized to support a wide range of SE tasks. In requirements engineering, they enable the elicitation, analysis, and validation of requirements by simulating human users interactions and processing natural language specifications [9], [10]. For code generation, role-specialized agents, such as planners, implementers, and reviewers, collaborate to produce code, integrate components, and ensure alignment to coding standards [11], [12]. Within the scope of quality assurance, multi-agent LLMs contribute to software testing through automated test case generation, execution, and analysis [13], [14]. Additionally, they support vulnerability detection by identifying and mitigating security flaws within software systems [13]. In software maintenance, they assist in monitoring runtime behavior, diagnosing faults, and generating appropriate patches or modifications [15], [16].

B. Multi-Agent Protocols

Recent efforts to formalize communication in multi-agent LLMs have led to the development of a diverse set of protocols. A recent survey [20] categorized these protocols along two dimensions: (1) context-oriented vs. inter-agent, depending on whether the focus is on managing internal context of agents or external coordination between them, and (2) general-purpose vs. domain-specific, depending on whether they are intended for broad use or tailored to a specific domain.

A significant advancement in general-purpose agent interoperability is Google's Agent-to-Agent (A2A) protocol. A2A establishes a modular communication framework where autonomous agents interact through structured HTTP-based APIs, using JSON-RPC 2.0 as the foundational message format. Each agent exposes a *run()* function and publishes a machine-readable Agent Card, a JSON-based declaration of its identity, supported endpoints, input/output modalities (e.g., text, image, file), authentication requirements, and capabilities. This approach enables dynamic discovery and seamless integration of agents across diverse environments, laying the groundwork for scalable and interoperable multi-agent LLMs.

III. METHODOLOGY

A. Methodology Overview

This section introduces the Software Engineering Multi-Agent Protocol (SEMAP), a structured methodology built on the insight that reliable multi-agent coordination requires the same core abstractions that underpin classical SE, namely components, interfaces, and transition logic. As shown in Fig. 1, SEMAP comprises three core principles, each grounded in foundational SE practices: (1) *Explicit behavioral contract modeling*, inspired by the principle of Design by Contract (DbC), formalizes agent responsibilities through pre-conditions and post-conditions. This reduces ambiguity and

mitigates under-specification at both role and task levels; (2) *Structured messaging*, drawing from typed interface design and schema-driven communication in SE, enforces semantically typed inter-agent messaging to ensure clarity, completeness, and coordination alignment; (3) *Lifecycle-guided execution with verification*, reflecting state machine-based workflow modeling and stage-wise testing in SE, structures collaboration around a task-specific lifecycle with embedded verification gates. This ensures output correctness and guards against premature or invalid termination.

B. Explicit Behavioral Contract Modeling

To address under-specification in multi-agent LLMs, the proposed methodology adopts a contract-oriented modeling paradigm inspired by DbC principle. In traditional SE, DbC establishes a formal agreement between software components, including pre-conditions define what must be true before execution, and post-conditions define what must hold afterward. Translating this into the multi-agent LLM systems context, each agent is modeled through an explicit behavioral contract, a verifiable schema that specifies the required input artifacts and expected output artifacts. Inputs capture the minimal artifacts the agent requires to operate meaningfully, such as task plans, peer feedback, or prior outputs, while outputs represent the agent's expected contributions, such as the code artifact generated by the Coder, or a review log produced by the Reviewer. This contract-oriented modeling formalizes agent responsibilities and role boundaries, reducing ambiguity across both design-time and runtime interactions. Formally, a behavioral contract $C \in \mathcal{C}$ is represented as a tuple as follows:

$$C = (\text{name}, \mathcal{I}_C, \mathcal{O}_C)$$

where:

- **name**: a role identifier (e.g., Reviewer);
- $\mathcal{I}_C$: set of required input artifacts (i.e., pre-conditions);
- $\mathcal{O}_C$: set of required output artifacts (i.e., post-conditions).

C. Structured Messaging

While behavioral contracts define the roles, inputs and outputs of agents, effective coordination also requires that inter-agent communication be semantically clear and complete. To address coordination misalignment in multi-agent LLMs, the proposed methodology adopts a structured messaging model that standardizes how information is exchanged between agents. Each message $M \in \mathcal{M}$ is formalized as:

$$M = (\text{sender}, \text{receiver}, \mathcal{C}_M)$$

where:

- **sender**: the identifier of the source agent;
- **receiver**: the identifier of the target agent;
- $\mathcal{C}_M$: a payload structured as a list of schema-designated objects (e.g., code, review_log, reviewer_comment).

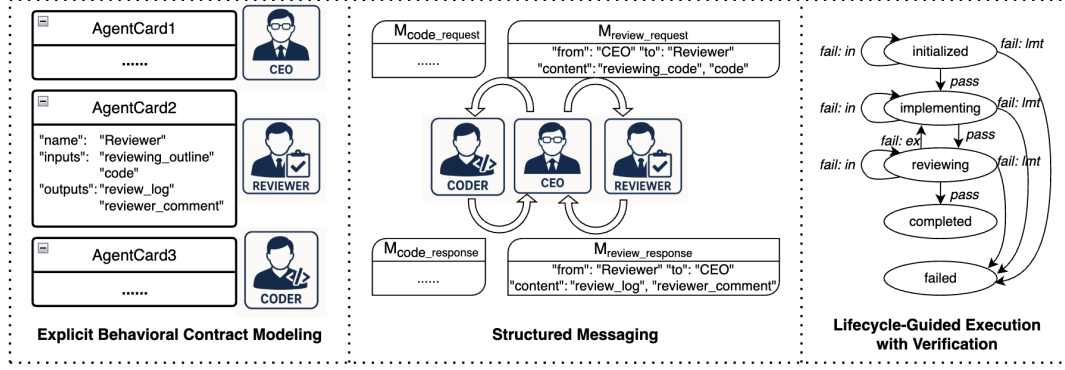


Fig. 1: Methodology Overview

D. Lifecycle-Guided Execution with Verification

While behavioral contracts and structured messaging ensure local correctness and coordination, they do not guarantee global task completion or output validity. To address this, the methodology introduces a lifecycle-guided execution model, which structures agent collaboration as a state machine with verification-driven transitions. This ensures that task progression is gated by validation and that failures can trigger appropriate recovery or reassignment actions. Formally, a task lifecycle is modeled as a finite state machine (FSM):

$$\mathcal{L} = (\mathcal{S}, \Sigma, \delta, s_0, \mathcal{F})$$

where:

- $\mathcal{S}$: a set of lifecycle stages (e.g., initialized, implementing, reviewing, completed, failed);
- Σ : verification outcomes (e.g., pass, fail);
- $\delta : \mathcal{S} \times \Sigma \rightarrow \mathcal{S}$: a transition function for the next stage;
- $s_0 \in \mathcal{S}$: the initial stage (typically initialized);
- $\mathcal{F} \subseteq \mathcal{S}$: terminal stages (e.g., completed, failed).

IV. PRELIMINARY EVALUATION

This section presents the evaluation of SEMAP, focusing on its effectiveness in reducing failures in multi-agent LLMs for SE tasks. The research questions (RQs) are set as follows:

- RQ1: To what extent does SEMAP mitigate failures across the three major categories, namely under-specification, coordination misalignment, and verification failure, compared to the baseline in different SE tasks?
- RQ2: How effectively does SEMAP enable consistent and stable failure reduction across collaboration rounds, particularly in minimizing recurrence in each category?

A. Experiment Settings

a) *Tasks and datasets*: The experiments include two representative SE tasks: software development and vulnerability detection, each evaluated using two comprehensive datasets.

Function-level development: In function-level development, agents solve problems from the HumanEval [21], each requiring the writing of a concise function that satisfies a given textual specification.

Deployment-level development: The deployment-level development task is based on ProgramDev [19], where agents develop a complete deployment starting from a one-sentence user requirement.

C/C++ vulnerability detection: This task uses devign100, a 100-sample subset of the Devign dataset [22], constructed by randomly selecting 50 vulnerable and 50 safe C/C++ functions. Each sample contains a function-level code snippet and a binary label indicating the presence or absence of a vulnerability. The code snippets are typically short (under 30 lines), enabling agents to focus on localized vulnerability patterns such as pointer misuse or unsafe memory operations.

Python vulnerability detection: This task uses vudenc100, a 100-sample dataset created by randomly sampling from the CVEFixes dataset [23]. A function is labeled as vulnerable if any of its lines are marked as such in the original line-level annotations. Similarly, the final dataset consists of 50 vulnerable and 50 safe Python functions. Compared to devign100, these code snippets are longer and structurally more complex, often containing real-world CVE patches with nested logic and broader reasoning scopes.

b) *Baseline and model settings*: The baseline system is implemented using the MetaGPT framework. For development tasks, it adopts a centralized CEO-style multi-agent architecture consisting of five agents: a CEO, a Planner, a Coder, a Reviewer, and a Tester. For vulnerability detection tasks, both the baseline and SEMAP use a decentralized three-agent settings consisting of an Auditor, a Critic, and a Tester.

Experiments are conducted using DeepSeek-V3-0324 and gpt-4.1-nano-2025-04-14 as the underlying agent models. For development tasks, SEMAP and the baseline are allowed up to five collaboration rounds per task. For vulnerability detection, the system executes a single round of analysis and voting.

c) *Evaluation metrics*: The LLM-as-a-Judge pipeline proposed in [19] is employed to categorize multi-agent LLMs failures, assigning each instance to one or more categories: specification issues, inter-agent misalignment, and task verification failures, using a separate model, gpt-4o-2024-08-06.

To evaluate RQ1, total number of failures in each category across different SE tasks, including both development and

vulnerability detection, is compared. To evaluate RQ2, change in failure counts change across collaboration rounds in development tasks, is compared between SEMAP and baseline.

B. Experiment Results

Results for RQ1: The total number of failures in each of the three major categories are reported in Table I (development tasks) and Table II (vulnerability detection), respectively. Across all datasets and model configurations, SEMAP consistently reduces the number of failures compared to the baseline.

In function-level development (HumanEval), SEMAP reduces the total number of failures by 64.1% with ChatGPT (from 256 to 92) and by 69.6% with DeepSeek (from 112 to 34). The largest reduction occurs in under-specification, where ChatGPT drops from 137 to 39 (71.5%), and DeepSeek from 63 to 17 (73.0%). Similarly, in deployment-level development (ProgramDev), SEMAP achieves a 12.6% reduction with ChatGPT (from 103 to 90) and a 56.7% reduction with DeepSeek (from 67 to 29), with the most notable improvement in under-specification for DeepSeek (39 to 18, a 53.8% decrease) and a complete elimination of inter-agent misalignment errors.

In vulnerability detection, SEMAP also shows consistent advantages over the baseline. On `devign100`, total failures decrease by 28.2% with ChatGPT (from 78 to 56) and by 8.3% with DeepSeek (from 48 to 44). On `vudenc100`, the reduction is 47.4% with ChatGPT (from 38 to 20) and 16.4% with DeepSeek (from 55 to 46). Unlike development tasks, the reductions here are more evenly distributed across the three failure categories, indicating SEMAP's balanced impact on decentralized workflows involving parallel agent collaboration.

Thus, RQ1 is answered: SEMAP consistently reduces failure counts across all categories and tasks, with the most significant improvements observed in under-specification during development, and more balanced reductions in vulnerability detection.

Results for RQ2. Figure 2 illustrates how failure counts evolve across development rounds, separated by different datasets and failure categories. Unlike RQ1, which evaluates end-to-end failure counts, RQ2 focuses on its changes during the iterative process, examining whether SEMAP promotes more stable and robust collaboration over repeated trials.

In the HumanEval dataset, SEMAP results in fewer failures across all development rounds compared to the baseline. As shown in Fig. 2a, SEMAP leads to a sharp and steady reduction in under-specification failures, dropping from 20 to 2 for ChatGPT-SEMAP and from 14 to 0 for DeepSeek-SEMAP by round 5. However, the baseline declines more gradually, with ChatGPT-Baseline ending at 9 failures and DeepSeek-Baseline at 2. Similar patterns also can be found in inter-agent misalignment (Fig. 2b) and task verification (Fig. 2c).

In the ProgramDev dataset, SEMAP continues to demonstrate improved round-wise behavior compared to the baseline. As shown in Fig. 2d, SEMAP significantly reduces under-specification failures across rounds, with DeepSeek-SEMAP dropping from 14 to 0 and ChatGPT-SEMAP from 16 to 3 by round 5. In contrast, the baseline systems converge more slowly and finish at 3 failures each, indicating limited

effectiveness in clarifying ambiguous task specifications. Also, similar results can be observed in inter-agent misalignment (Fig. 2e) and task verification (Fig. 2f).

Compared to the baseline, SEMAP not only reduces failure counts more effectively, but also exhibits a more stable downward trend across rounds, whereas baseline failures often reappear after temporary drops. The underlying mechanisms contributing to this stability are further discussed later.

Thus, RQ2 is answered: SEMAP promotes more reliable and consistent behavior across collaboration rounds, reducing the recurrence of specification, coordination, and verification failures in both function-level and deployment-level development.

V. CONCLUSION AND FUTURE WORKS

This paper presents SEMAP, a protocol-layer methodology for addressing common failures in multi-agent LLMs, including under-specification, coordination misalignment, and inadequate verification. SEMAP instantiates three SE-informed principles, behavioral contracts, structured messaging, and lifecycle-guided execution with verification, supporting both centralized and decentralized workflows. Empirical results across diverse SE tasks demonstrate that SEMAP substantially improves system robustness. In function-level and deployment-level code development, it achieves up to 69.6% and 56.7% reductions in total failures, with the greatest gains observed in mitigating under-specification and coordination issues. In vulnerability detection, SEMAP consistently reduces failure rates across Python and C/C++ tasks up to 47.4%, while promoting better agent alignment in decentralized workflows.

To strengthen validity, future experiments will be scaled to larger datasets, agent populations, and longer workflows, and compared against more baselines, including single-agent LLMs and domain-specific detectors. Ablation studies will isolate the impact of contracts, messaging, and lifecycle control. Future work also includes measuring resource overhead, enabling cross-agent tool use, adding formal protocol correctness verification, and releasing artifacts for reproducibility.

REFERENCES

- [1] SlashData, "Global developer population trends 2025: How many developers are there?" 2025, accessed: 2025-05-15. [Online]. Available: <https://www.slashdata.co/post/global-developer-population-trends-2025-how-many-developers-are-there>
- [2] J. Cai *et al.*, "Exploring the improvement of evolutionary computation via large language models," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2024, pp. 83–84.
- [3] J. Fan *et al.*, "Coding latent concepts: a human and llm-coordinated content analysis procedure," *Communication Research Reports*, vol. 41, no. 5, pp. 324–334, 2024.
- [4] J. Li *et al.*, "Generative ai for self-adaptive systems: State of the art and research roadmap," *ACM Transactions on Autonomous and Adaptive Systems*, vol. 19, no. 3, pp. 1–60, 2024.
- [5] Y. Sun *et al.*, "Semirald: A semi-supervised hybrid language model for robust anomalous log detection," *Information and Software Technology*, vol. 183, p. 107743, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584925000825>
- [6] Z. Yang *et al.*, "Exploring and unleashing the power of large language models in automated code translation," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, Jul. 2024. [Online]. Available: <https://doi.org/10.1145/3660778>
- [7] Z. Mao *et al.*, "Hybrid privacy policy-code consistency check using knowledge graphs and llms," 2025. [Online]. Available: <https://arxiv.org/abs/2505.11502>

TABLE I: Results of RQ1: Development Tasks

Failure Category	HumanEval						ProgramDev					
	GPT-4.1-nano			Deepseek-V3			GPT-4.1-nano			Deepseek-V3		
	Baseline	SEMAP	$\Delta\%$	Baseline	SEMAP	$\Delta\%$	Baseline	SEMAP	$\Delta\%$	Baseline	SEMAP	$\Delta\%$
Under-specification	137	39	71.5	63	17	73.0	48	46	4.1	39	18	53.8
Inter-Agent Misalignment	27	5	81.5	10	3	70.0	9	9	0.0	8	0	100.0
Task Verification	92	48	47.8	39	14	64.1	46	35	23.9	20	11	45.0
Total	256	92	64.1	112	34	69.6	103	90	12.6	67	29	56.7

TABLE II: Results of RQ1: Vulnerability Detection Tasks

Failure Category	devign100 (C/C++)						vudenc100 (Python)					
	GPT-4.1-nano			Deepseek-V3			GPT-4.1-nano			Deepseek-V3		
	Baseline	SEMAP	$\Delta\%$	Baseline	SEMAP	$\Delta\%$	Baseline	SEMAP	$\Delta\%$	Baseline	SEMAP	$\Delta\%$
Under-specification	12	9	25.0	5	4	20.0	4	3	25.0	16	15	6.3
Inter-Agent Misalignment	33	24	27.2	20	18	10.0	14	11	21.3	16	11	31.3
Task Verification	33	23	30.3	23	20	13.0	10	6	40.0	23	20	13.0
Total	78	56	28.2	48	44	8.3	38	20	47.4	55	46	16.4

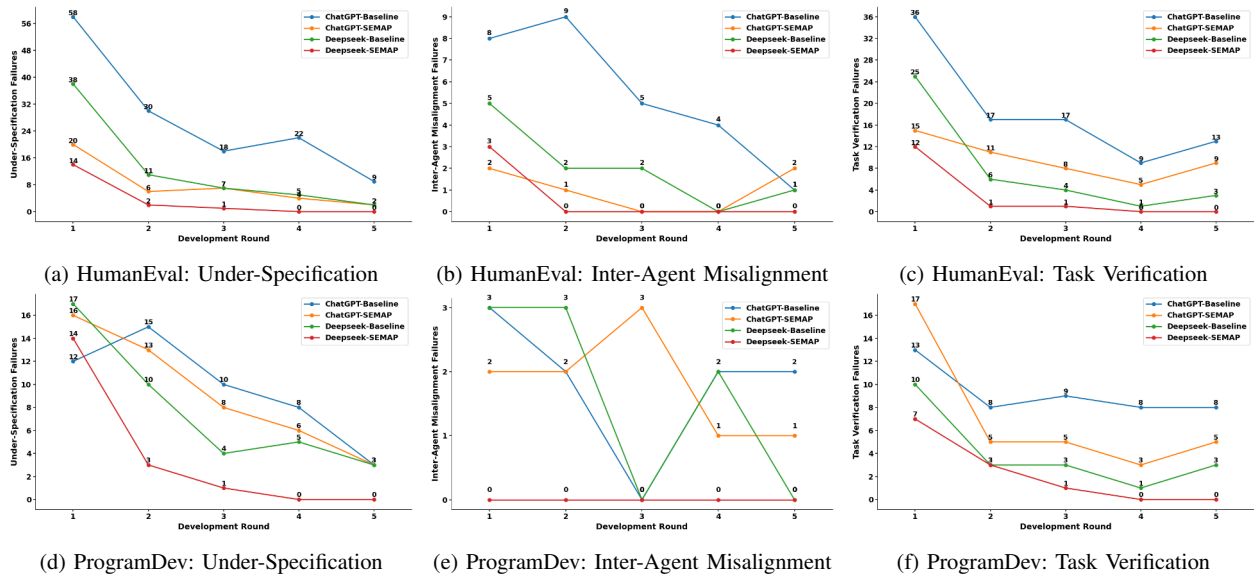


Fig. 2: Results of RQ2: Failures v.s. Rounds in Development Tasks

- [8] J. He *et al.*, “Llm-based multi-agent systems for software engineering: Literature review, vision and the road ahead,” *ACM Transactions on Software Engineering and Methodology*, 2024.
- [9] D. Jin *et al.*, “Mare: Multi-agents collaboration framework for requirements engineering,” *arXiv preprint arXiv:2405.03256*, 2024.
- [10] M. Ataei *et al.*, “Elicitron: An llm agent-based simulation framework for design requirements elicitation,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.16045>
- [11] H. Zhang *et al.*, “A pair programming framework for code generation via multi-plan exploration and feedback-driven refinement,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1319–1331.
- [12] D. Zan *et al.*, “Codes: Natural language to code repository via multi-layer sketch,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.16443>
- [13] Z. Mao *et al.*, “Multi-role consensus through llms discussions for vulnerability detection,” in *2024 IEEE 24th International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*. IEEE, 2024, pp. 1318–1319.
- [14] S. Hu *et al.*, “Large language model-powered smart contract vulnerability detection: New perspectives,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.01152>
- [15] H. Wang *et al.*, “Intervenor: Prompting the coding ability of large language models with the interactive chain of repair,” *arXiv preprint arXiv:2311.09868*, 2023.
- [16] W. Tao *et al.*, “Magis: Llm-based multi-agent framework for github issue resolution,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.17927>
- [17] D. Cohen *et al.*, “An introduction to agile methods,” *Advances in computers*, vol. 62, no. 03, pp. 1–66, 2004.
- [18] K. Petersen *et al.*, “The waterfall model in large-scale development,” in *International Conference on Product-Focused Software Process Improvement*. Springer, 2009, pp. 386–400.
- [19] M. Cemri *et al.*, “Why do multi-agent llm systems fail?” *arXiv preprint arXiv:2503.13657*, 2025.
- [20] Y. Yang *et al.*, “A survey of ai agent protocols,” *arXiv preprint arXiv:2504.16736*, 2025.
- [21] M. Chen *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [22] Y. Zhou *et al.*, “Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 10 197–10 207.
- [23] H.-C. Tran *et al.*, “Detectvul: A statement-level code vulnerability detection for python,” *Future Generation Computer Systems*, p. 107504, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X24004680>